

# 作业题 14

## 题目

以  $x_{n+1} = \lambda \sin(\pi x_n)$  为迭代方程进行迭代：

(1)画出系统状态随参数  $\lambda$  的变化图，要求在图中体现出定值状态、倍周期分叉和混沌状态；

(2)列出各个倍周期分叉处的  $\lambda$  值，求相应的 *Feigenbaum* 常数。

## 代码

```
1. import numpy as np
2. import matplotlib.pyplot as plt
3.
4. aaaaaaaaa = 16807
5. m = 2147483647
6.
7. def azmodm(z):
8.     global m, aaaaaaaaa
9.     q = int(m / aaaaaaaaa)
10.    r = m % aaaaaaaaa
11.    tmp = aaaaaaaaa * (z % q) - r * int(z / q)
12.    if tmp >= 0:
13.        return tmp
14.    else:
15.        return tmp + m
16.
17. bbbssbbb = 0
18. aaassaaa = 0
19. cccssccc = 0
20.
21. def random01():
22.     global bbbssbbb, aaassaaa, m, cccssccc
```

```

23.     z1 = 666955
24.     z2 = 816815
25.     z3 = 116560
26.     z4 = 4813144
27.     if aaasssaaa == 0 and cccsssccc == 0:
28.         bbbsssbbb = z1
29.     if aaasssaaa == m - 10000000 and cccsssccc == 0:
30.         bbbsssbbb = z2
31.         aaasssaaa = 1
32.         cccsssccc = 1
33.     if aaasssaaa == m - 10000000 and cccsssccc == 1:
34.         bbbsssbbb = z3
35.         aaasssaaa = 1
36.         cccsssccc = 2
37.     if aaasssaaa == m - 10000000 and cccsssccc == 2:
38.         bbbsssbbb = z4
39.         aaasssaaa = 1
40.         cccsssccc = 3
41.     bbbsssbbb = azmodm(bbbsssbbb)
42.     aaasssaaa += 1
43.     return bbbsssbbb / m
44.
45.
46.
47. def xnn(x, lam): # 迭代方程
48.     return lam * np.sin(np.pi * x)
49.
50.
51. def lx(x0, lam, lxs):# 计算最终状态
52.     results = [x0]
53.     for _ in range(lxs):
54.         x0 = xnn(x0, lam)
55.         results.append(x0)
56.     return np.array(results)
57.
58.
59. def findx(arr,i): ##对重复 x 进行合并
60.     if len(np.unique(arr)) == 1:
61.         return 1
62.     # 对数组进行排序
63.     sorted_arr = np.sort(arr)
64.     # 找到相邻元素之间的差值
65.     differences = np.diff(sorted_arr)
66.     threshold = 0.1**(i+1) ##间距按标度律递增

```

```

67.     # print(differences)
68.
69.     # 使用差值小于等于阈值的索引，将相邻元素合并
70.     combined_indices = np.where(differences > threshold)[0]
71.
72.     # 构建包含不同元素的新数组
73.     unique_elements = [sorted_arr[0]] + [sorted_arr[i + 1] for i in combined_indices]
74.     difference = np.diff(unique_elements)
75.     d=np.around(difference,6)
76.     d=np.unique(d)
77.     # print(d)
78.     return len(unique_elements) , d
79.
80.
81.
82. mm=[]
83. lamm=[]
84. deta=[]
85. dd=[]
86. arf=[]
87. mm.append(1)
88.
89. def F(lams, lxs, jump):
90.     global mm
91.     global lamm
92.     global deta
93.     global dd
94.     global arf
95.     all_x = []
96.     all_lam = []
97.     i=0
98.     for lam in lams:
99.         x = 2*random01()# 初始条件
100.         for _ in range(jump):      # 计算每个参数值下的状态
101.             x = xnn(x, lam)
102.             xn = lx(x, lam, lxs)
103.             xn=np.append(xn,-xn)
104.             n , d =findx(xn,i)
105.             # print(d)
106.             if(n==mm[i]*2 ):
107.                 mm.append(n)
108.                 i+=1
109.                 lamm.append(lam)

```

```

110.         # dd.append()
111.         # d=np.unique(d)  ##去掉重复值
112.         # print(d)
113.         # print(i)
114.         if(i>=2):
115.             dd.append(d[1-i])  ##取第(i-1)大的间隔
116.             # print(d)
117.             # print(dd)
118.         if(i>=3):
119.             deta.append((lamm[i-2]-lamm[i-3])/(lamm[i-1]-lamm[i-2]))
120.             arf.append(dd[i-3]/dd[i-2])
121.         all_x.extend(xn)
122.         # all_x.extend(-xn)
123.         all_lam.extend([lam] * len(xn))
124.         # all_lam.extend([lam] * len(xn))
125.
126.         print("分叉数 m:")
127.         print(mm)
128.         print("分叉值")
129.         print(lamm)
130.         print("分叉间隔比值:")
131.         print(deta)
132.         print("分叉间距:")
133.         print(dd)
134.         print("间距比:")
135.         print(arf)
136.
137.         plt.figure(figsize=(10, 6))
138.         plt.scatter(all_lam, all_x, s=0.00001, c='black')
139.         plt.xlabel('λ')
140.         plt.ylabel('x')
141.         plt.show()
142.
143.
144.
145.     lams=np.linspace(0.3, 1.2, 7000)  # 参数范围
146.     lxs=400  # 每个参数值的迭代次数
147.     jump =10000  # 跳过的迭代次数
148.     F(lams, lxs, jump)

```

大致算法：lam 从 (0.3, 1.2) 取 7000 个值，对每个 lam，随机抽取一个初始值，迭代 1000 次达到周期或混沌状态，再计算之后的 400 个值，这 400 个值基本包含了所有周期值（或非周期，是混沌状态）。画图便将这 400 个值对应每个 lam 描点。

计算分叉间隔比值：对 400 个值排序，计算间隔，将间隔小于 threshold 的点视为一个

点，此处  $i$  表示分叉次数。根据标度律（每次分叉的尺度之比为常数），这里应该用  $\text{threshold} = 0.1^{(i+1)}$ ，而不应该设为常数。合并以后，重新计算间隔，返回合并后的点数量和间隔数组  $d$ （用于计算分叉间距），每当检测到点的数量倍增时，记录  $\text{lam}$  分叉值和对应的分叉数（即点的数量）。最后计算分叉间隔比值  $\text{deta}$ 。

计算分叉间距和间距比：每次检测到点的数量倍增后，记录间隔数组  $d$  中第  $(i-1)$  大的值，作为分叉间距（此处分叉间距和讲义中不同，这里计算的是分叉点处的分叉间距）。最后计算分叉间距比  $\text{arf}$ 。

## 结果

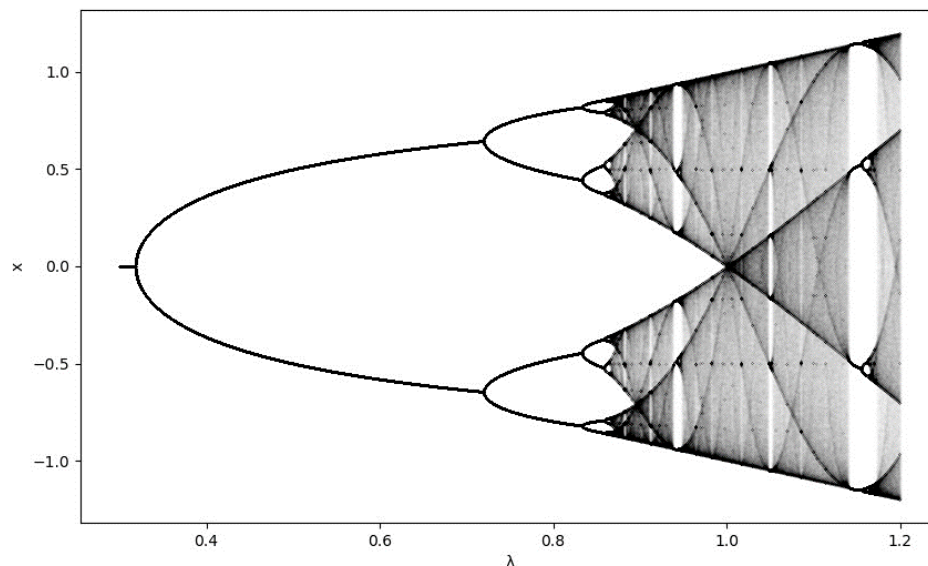


图 1

打印结果：

分叉数  $m$ :

[1, 2, 4, 8, 16, 32, 64, 128]

分叉值

[0.31967423917702525, 0.720102871838834, 0.8333904843549078, 0.8587226746678096, 0.864123446206601, 0.8652807543934848, 0.86553793399057]

分叉间隔比值:

[3.5346197502837664, 4.4720812182741225, 4.6904761904760806, 4.666666666666667, 4.5000000000000259]

分叉间距:

[1.277256, 0.368344, 0.1418, 0.055819, 0.038478, 0.016448]

间距比:

[3.4675629303042808, 2.5976304654442877, 2.5403536430247766, 1.4506731119081033, 2.339372568093385]

由结果可知，分叉数到了 128 就结束了，原因可能有：

- 1、lams 数组的间隔限制，导致后面分叉间隔小于数组间隔，无法得到分叉点，加密数组可以解决此问题
- 2、另一方面是混沌本身导致的分叉数急剧增加，利用间隔小于 `threshold` 的点合并的方法失效了，因此这里应该更加细致地考虑间隔公式里的常数 **0.1**。

另外，128 分叉这个点是否真的为分叉点存疑，由结果看到，这里的分叉间隔比值为 4.5000，小于标准值，因此 64-→128 的分叉间隔偏大，可能是因为算法精度不够导致（上面两点的），也可能此时已经到达混沌状态，出现 3，6、、、周期。

间距比的结果大致和标准值一致，但出现 1.450 这样的误差较大的值，可能是间距选取有误。